INDIAN INSTITUTE OF TECHNOLOGY ROORKEE

Roorkee, Uttarakhand – 247667, India

Blockchain Technology: Foundations, Mechanisms, and Future Directions

A Technical Report

Submitted in partial fulfilment of the requirements for Technical Communication (HSN–301)

Group 21

Jai Bhadu

Ayushka Garg

Anmol Goklani

Nitin Raj

Madhav Deorah

April 2025

Statement of Contributions

This report is the combined effort of all five members of Group 21. The individual contributions of each author are detailed below.

Author	**Contribution**
Jai Bhadu	Authored Chapter 1 (Introduction) covering the motivation behind blockchain, the comparison between centralised and decentralised systems, and a high-level overview of the technology. Also contributed the PNB Scam case study illustrating failures of centralised trust.
Ayushka Garg	Authored Chapter 2 (Preliminaries and Background) documenting the pre-2008 historical foundations of blockchain including hash chains, Merkle trees, and Bit Gold, as well as the detailed analysis of the Bitcoin Whitepaper covering transactions, timestamp servers, and Proof of Work.
Anmol Goklani	Authored Chapter 3 (Consensus Mechanisms and Scalability Engineering) covering the Byzantine Generals Problem, types of consensus protocols (PoW, PoS, PBFT), and scalability approaches including sharding, Layer-2 solutions, and rollups.
Nitin Raj	Authored Chapter 4 (Advanced Directions and Applications) covering smart contract evolution, zero-knowledge proofs, cross-chain interoperability, and real-world applications in finance, supply chain, healthcare, and digital identity.
Madhav Deorah	Authored Chapter 5 (Challenges and the Blockchain Trilemma) detailing scalability bottlenecks, energy consumption, latency, transaction fees, and the fundamental security–scalability–decentralisation trade-off. Also contributed the Conclusion (Chapter 6).

Table 1: Individual contributions of each team member.

All team members cross-checked each other's chapters, verified figures and tables for accuracy, and jointly compiled the bibliography.

Acknowledgements

We would like to express our sincere gratitude to the faculty of the Technical Communication course (HSN–301) at IIT Roorkee for providing us with the opportunity to explore and present on blockchain technology. Their guidance in structuring technical arguments and presenting complex ideas to a broad audience was invaluable.

We also thank our peers for their constructive feedback during the presentation that informed several improvements in this report. Finally, we acknowledge the authors of the foundational research papers—particularly Satoshi Nakamoto, Stuart Haber, W. Scott Stornetta, and Nick Szabo—whose pioneering work forms the intellectual backbone of this report.

Abstract

Blockchain technology represents a fundamental shift in how digital trust is established. Rather than relying on a central authority—such as a bank, government, or corporation—to verify transactions and maintain records, blockchain enables a network of independent participants to collectively agree on a single, tamper-proof version of the truth. This is achieved through a combination of cryptographic hashing, distributed consensus protocols, and economic incentives.

This report provides a comprehensive introduction to blockchain technology, tracing its origins from early cryptographic time-stamping work in the 1990s through the landmark Bitcoin whitepaper of 2008 and into modern developments such as smart contracts, zero-knowledge proofs, and cross-chain interoperability. We examine the core mechanisms that make blockchains work—including digital signatures, Merkle trees, Proof of Work, and Proof of Stake—and analyse the fundamental engineering trade-offs captured by the so-called *Blockchain Trilemma* among security, scalability, and decentralisation.

We survey real-world applications across finance, supply chain management, healthcare, and digital identity, and discuss the open challenges that must be addressed before blockchain can achieve mainstream adoption. The report concludes with an outlook on future directions, including energy-efficient consensus mechanisms, integration with artificial intelligence and the Internet of Things, and the vision of a fully interoperable multi-chain ecosystem.

Keywords: Blockchain, Bitcoin, Distributed Ledger, Consensus Mechanisms, Proof of Work, Smart Contracts, Scalability, Decentralisation.

Contents

Statement of Contributions **1**

Acknowledgements **2**

Abstract **3**

List of Figures **8**

List of Tables **9**

List of Abbreviations **10**

List of Abbreviations **10**

1 Introduction **11**

1.1 Motivation 11

1.2 Centralised vs. Decentralised Systems 12

1.3 What Is a Blockchain? 13

1.4 Our Contributions 13

1.5 Report Organisation 14

2 Preliminaries and Background **15**

2.1 Pre-2008 Foundations 15

2.1.1 Hash Chains (Haber & Stornetta, 1991) 15

2.1.2 Merkle Trees (1992) 17

2.1.3 Bit Gold (Nick Szabo, 1998) 17

2.2 The Bitcoin Whitepaper 18

2.2.1 The Problem with Traditional Systems 18

2.2.2 The Double-Spending Problem 18

2.2.3 Bitcoin Transactions 18

2.2.4 The Timestamp Server 19

2.2.5 Proof of Work 19

3 Consensus Mechanisms and Scalability Engineering 21
3.1 The Consensus Problem . . . 21
3.2 Types of Consensus Mechanisms . . . 22
3.2.1 Proof of Work (PoW) . . . 22
3.2.2 Proof of Stake (PoS) . . . 23
3.2.3 Practical Byzantine Fault Tolerance (PBFT) . . . 23
3.2.4 Comparison of Consensus Mechanisms . . . 23
3.3 Scalability Engineering . . . 24
3.3.1 The Scalability Problem . . . 24
3.3.2 Sharding . . . 24
3.3.3 Layer-2 Solutions . . . 25
3.3.4 Rollups . . . 25
3.3.5 Larger Block Size . . . 26
3.3.6 Summary of Scaling Approaches . . . 26

4 Advanced Directions and Applications of Blockchain 27
4.1 Smart Contract Evolution . . . 27
4.2 Privacy Enhancements: Zero-Knowledge Proofs . . . 28
4.3 Cross-Chain Interoperability . . . 28
4.4 Applications of Blockchain . . . 29
4.4.1 Finance and Decentralised Finance (DeFi) . . . 29
4.4.2 Supply Chain Management . . . 29
4.4.3 Healthcare . . . 30
4.4.4 Digital Identity . . . 30
4.4.5 Summary of Applications . . . 31

5 Challenges in Blockchain and the Blockchain Trilemma 32
5.1 Scalability Issues . . . 32
5.2 High Energy Consumption . . . 33
5.3 Latency and Speed . . . 33
5.4 High Transaction Fees . . . 33
5.5 Storage Bloat and Data Availability . . . 34
5.6 The Blockchain Trilemma . . . 34

6 Conclusion and Future Work 36
6.1 Conclusion . . . 36
6.2 Future Directions . . . 37
6.2.1 Scalable Blockchain Systems . . . 37
6.2.2 Energy-Efficient Consensus . . . 37
6.2.3 Cross-Chain Ecosystems . . . 37

6.2.4 Integration with AI and IoT 37
6.2.5 Regulatory and Governance Frameworks 38

A Supplementary Code: Simple Blockchain in Python 41

B Blockchain Structure Diagram 45

C Glossary of Blockchain Terms 46

List of Figures

1.1 Centralised vs. decentralised network topologies. In a centralised system (left), all clients depend on a single server—a single point of failure. In a decentralised system (right), every node connects to multiple peers, so the network remains functional even if several nodes go offline. . . 12

2.1 Timeline of key milestones leading to the creation of Bitcoin in 2008. 16

2.2 A Merkle tree with four transactions. Each leaf is the hash of a transaction; each internal node is the hash of its children. The Merkle root at the top provides a compact, tamper-evident summary of all transactions in the block. 17

2.3 Bitcoin transaction chain. Each transaction contains the hash of the previous transaction and the new owner's public key, signed by the previous owner. This creates a verifiable chain of ownership. 19

2.4 The blockchain as a timestamp server. Each block contains a batch of transactions, a hash of its own contents, and the hash of the previous block. Altering any block would change its hash and break all subsequent links. 19

2.5 The Proof of Work mining process. A miner repeatedly tries different nonces until the hash of the block falls below the target. Finding a valid nonce is hard; verifying it is easy. 20

3.1 Sharding: the network is partitioned into independent shards that process transactions in parallel. Cross-shard communication enables transactions that span multiple shards. 25

4.1 How a smart contract works. An external event triggers the contract, which evaluates its conditions and, if they are met, automatically executes the agreed-upon action and records the result on the blockchain. 28

4.2 Conceptual illustration of a zero-knowledge proof. The prover demonstrates knowledge of a secret without ever revealing the secret itself. . 28

5.1 The Blockchain Trilemma. A blockchain system can usually improve only two of the three properties at the same time: decentralisation, security, and scalability. Improving one often affects another. 35

6.1 Four key future directions for blockchain technology. 38

B.1 Anatomy of a blockchain block. The block header contains metadata (including the hash link to the previous block and the Merkle root of all transactions). The block body contains the actual transactions. . . 45

List of Tables

1 Individual contributions of each team member. 1

1.1 Comparison of centralised and decentralised systems. 12

3.1 Comparison of Proof of Work, Proof of Stake, and PBFT consensus mechanisms. 24

3.2 Throughput comparison: blockchain networks vs. traditional payment systems [1]. 24

3.3 Summary of blockchain scalability approaches and their trade-offs. . . 26

4.1 Summary of blockchain applications across different domains. 31

5.1 How different blockchain systems handle the Blockchain Trilemma. . 35

List of Abbreviations

AI Artificial Intelligence.

CBS Core Banking System.

DApp Decentralized Application.

DeFi Decentralized Finance.

IoT Internet of Things.

LoU Letter of Undertaking.

PBFT Practical Byzantine Fault Tolerance.

PoS Proof of Stake.

PoW Proof of Work.

SSI Self-Sovereign Identity.

TPS Transactions Per Second.

ZKP Zero-Knowledge Proof.

Chapter 1

Introduction

1.1 Motivation

For most of modern history, whenever two people who did not know each other wanted to exchange something of value—money, property records, medical data—they needed a trusted middleman to make it work. Banks verify that you have enough money before letting you send a payment. Government registries confirm who owns a piece of land. Hospitals keep your medical records behind locked systems. These centralised systems work well *most of the time*, but they have a fundamental weakness: they concentrate enormous power and responsibility in a single place. If that single place is compromised—by a technical failure, a corrupt employee, or a sophisticated attacker—the consequences can be devastating.

Consider the Punjab National Bank (PNB) fraud of 2018, one of the largest banking scams in Indian history. A small group of employees, in collusion with jeweller Nirav Modi, issued fraudulent Letter of Undertakings (LoUs) worth over INR 13,000 crore (approximately USD 1.8 billion). These letters were never recorded in the bank's Core Banking System (CBS), meaning the fraud went undetected for years [2]. The scam was possible precisely *because* the system was centralised: a handful of insiders could bypass the controls that everyone else trusted implicitly.

Blockchain technology offers a radically different approach. Instead of placing trust in a single institution, it distributes trust across an entire network of participants. Every transaction is recorded on a shared ledger that anyone can verify, no single party can alter, and no central point of failure can bring down. In the words of the technology's most famous proponent, Satoshi Nakamoto, the goal is "a system for electronic transactions without relying on trust" [3].

1.2 Centralised vs. Decentralised Systems

To appreciate what blockchain does differently, it helps to first understand the two paradigms it bridges. Figure 1.1 illustrates the structural difference.

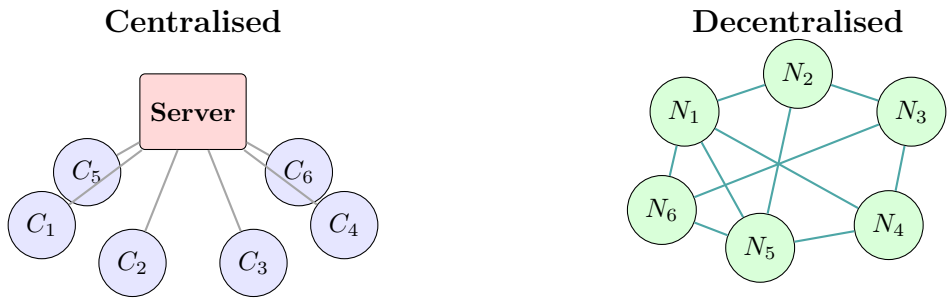

Figure 1.1: Centralised vs. decentralised network topologies. In a centralised system (left), all clients depend on a single server—a single point of failure. In a decentralised system (right), every node connects to multiple peers, so the network remains functional even if several nodes go offline.

Centralised systems are architectures in which a single authority controls data storage, transaction validation, and access permissions. Examples include traditional banks, social media platforms such as Facebook (Meta), and cloud services like Google. While efficient, they suffer from a critical vulnerability: a single point of failure. If the central server is compromised, the entire system is affected.

Decentralised systems distribute control across many independent nodes. There is no single entity that can unilaterally alter, censor, or shut down the network. Data is replicated across nodes, making the system highly resilient to failures and attacks. Blockchain is the most prominent technology enabling such decentralisation for transactional data.

Table 1.1 summarises the key differences.

Property	Centralised	Decentralised
Control	Single authority	Distributed among all participants
Point of failure	Single (the central server)	None (network tolerates node failures)
Transparency	Opaque to end users	Fully auditable by anyone
Trust model	Trust in the institution	Trust in the protocol and mathematics
Examples	Banks, Google, Facebook	Bitcoin, Ethereum

Table 1.1: Comparison of centralised and decentralised systems.

1.3 What Is a Blockchain?

At its simplest, a **blockchain** is a continuously growing list of records—called *blocks*—that are linked together using cryptographic hashes. Each block contains a batch of transactions, a timestamp, and the hash of the previous block, forming an unbroken chain stretching all the way back to the very first block (the *genesis block*).

The key properties that make this simple structure so powerful are:

1. **Immutability.** Once data is recorded in a block and the block is added to the chain, it cannot be changed or deleted without redoing all subsequent work—a computationally prohibitive task.

2. **Decentralisation.** No single authority controls the blockchain. Instead, a network of nodes collectively maintains the ledger.

3. **Transparency.** The ledger is public; anyone can verify any transaction at any time.

4. **Resilience.** Because the same data exists on thousands of nodes worldwide, the system has no single point of failure.

Put differently: in a blockchain system, *trust is enforced by the system itself*—through transparency and mathematics—rather than by people or institutions.

1.4 Our Contributions

This report provides a self-contained, accessible introduction to blockchain technology. Our specific contributions are:

- A clear explanation of the historical foundations of blockchain, from hash chains (1991) through Bit Gold (1998) to the Bitcoin whitepaper (2008)—see Chapter 2.

- A detailed walkthrough of how Bitcoin transactions, timestamp servers, and Proof of Work operate, written for readers without a prior background in cryptography—see Chapter 2.

- A comparative analysis of consensus mechanisms (PoW, PoS, PBFT) and scalability solutions (sharding, Layer 2, rollups)—see Chapter 3.

- A survey of real-world blockchain applications and advanced directions including smart contracts, zero-knowledge proofs, and cross-chain communication—see Chapter 4.

- A discussion of the open challenges facing blockchain adoption and a formal treatment of the Blockchain Trilemma—see Chapter 5.

1.5 Report Organisation

The remainder of this report is organised as follows:

Chapter 2 presents the historical and technical foundations: pre-2008 work, the Bitcoin protocol, and core cryptographic primitives.

Chapter 3 examines consensus mechanisms and scalability engineering.

Chapter 4 surveys advanced directions and real-world applications.

Chapter 5 discusses open challenges and the Blockchain Trilemma.

Chapter 6 concludes the report and outlines future research directions.

Chapter 2

Preliminaries and Background

Declaration of Contributions: This chapter was primarily authored by **Ayushka Garg**, who researched and wrote the sections on pre-2008 historical work (hash chains, Merkle trees, Bit Gold), the Bitcoin Whitepaper analysis, and the Bitcoin transaction model. **Jai Bhadu** contributed the introductory definitions and the section on blockchain's core properties. All team members reviewed the chapter for accuracy.

This chapter lays the groundwork needed to understand the rest of the report. We begin with the pre-2008 intellectual history that made blockchain possible, then provide a careful walkthrough of the Bitcoin whitepaper—the document that brought all these ideas together into a working system.

2.1 Pre-2008 Foundations

Blockchain did not appear out of thin air in 2008. It was the culmination of decades of research in cryptography, distributed systems, and digital currency design. Figure 2.1 shows the key milestones.

2.1.1 Hash Chains (Haber & Stornetta, 1991)

In 1991, Stuart Haber and W. Scott Stornetta published a paper addressing a deceptively simple question: *how can you prove that a digital document existed at a particular point in time and has not been altered since?* [4]

Their solution was the **hash chain**. Each document is run through a cryptographic hash function—a mathematical function that takes any input and produces a fixed-size output (the "hash" or "digest"). Crucially, even a tiny change to the input produces a completely different hash. By linking each new document's hash to the hash of the

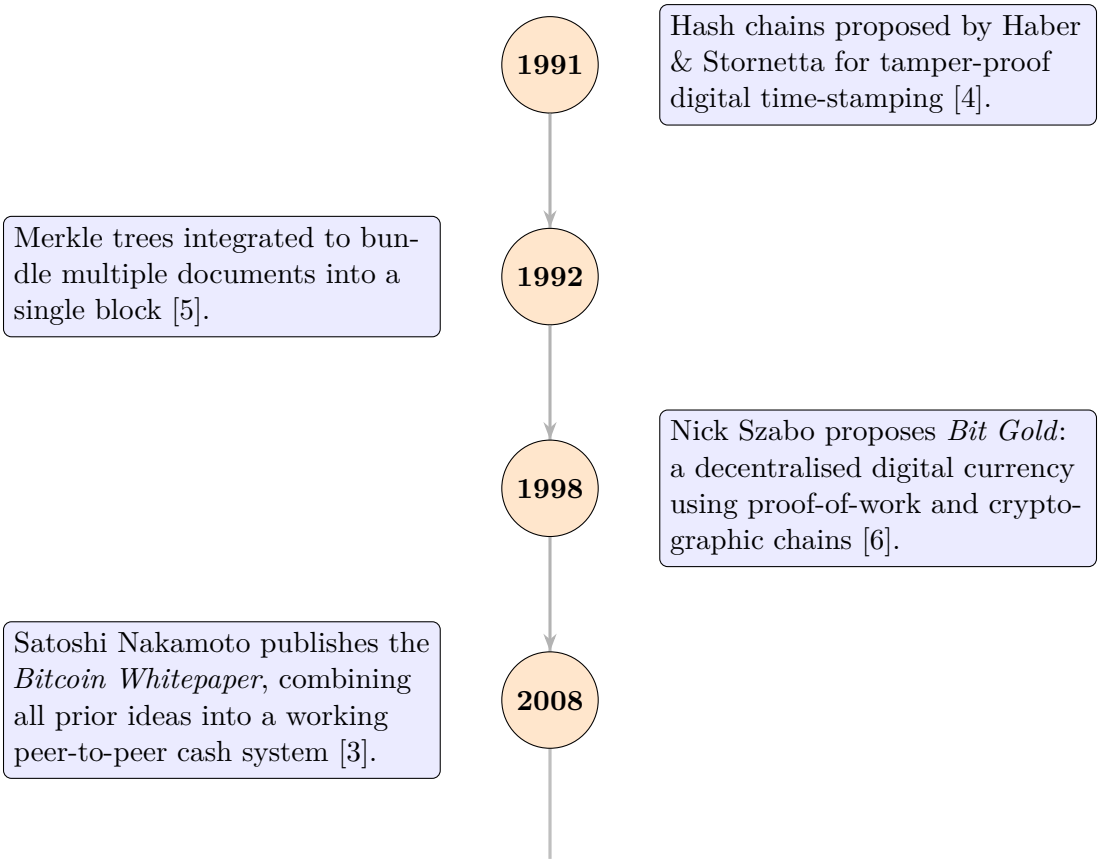

Figure 2.1: Timeline of key milestones leading to the creation of Bitcoin in 2008.

previous document, they created a chain where altering any single entry would break all subsequent links.

Definition 2.1 (Cryptographic Hash Function)**.** *A cryptographic hash function* $H : \{0,1\}^* \to \{0,1\}^n$ *maps an arbitrary-length input to a fixed-length output and satisfies three properties:*

1. ***Pre-image resistance:*** *Given* h*, it is computationally infeasible to find* m *such that* $H(m) = h$*.*

2. ***Second pre-image resistance:*** *Given* m_1*, it is infeasible to find* $m_2 \neq m_1$ *such that* $H(m_1) = H(m_2)$*.*

3. ***Collision resistance:*** *It is infeasible to find* any *pair* (m_1, m_2) *with* $m_1 \neq m_2$ *and* $H(m_1) = H(m_2)$*.*

This idea—linking records cryptographically so that tampering with any one record is immediately detectable—is the intellectual seed from which blockchain grew.

2.1.2 Merkle Trees (1992)

In 1992, Haber and Stornetta improved their scheme by incorporating **Merkle trees** [5], a data structure invented by Ralph Merkle [7]. Instead of hashing documents one-by-one into a linear chain, multiple documents are organised into a binary tree. Each leaf node is the hash of a document, and each internal node is the hash of its two children. The single hash at the top of the tree—the *Merkle root*—summarises all the documents below it.

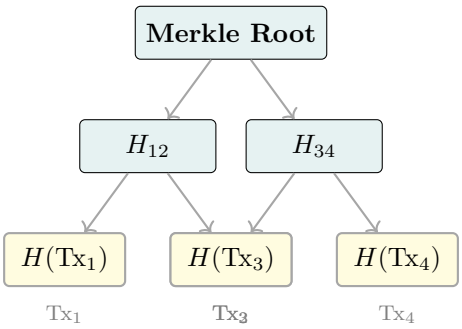

Figure 2.2: A Merkle tree with four transactions. Each leaf is the hash of a transaction; each internal node is the hash of its children. The Merkle root at the top provides a compact, tamper-evident summary of all transactions in the block.

The Merkle tree provides two important benefits:

- **Efficiency:** To verify that a specific transaction is included in a block, you only need a small number of hashes (logarithmic in the total number of transactions), not the entire block.
- **Integrity:** Changing any single transaction changes its leaf hash, which propagates up the tree and changes the Merkle root, making tampering immediately detectable.

2.1.3 Bit Gold (Nick Szabo, 1998)

In 1998, computer scientist Nick Szabo proposed *Bit Gold* [6], an ambitious design for a fully decentralised digital currency. Bit Gold combined several ideas:

- A **proof-of-work** mechanism requiring computational effort to create new units of currency (analogous to the effort required to mine physical gold).
- **Cryptographic chains** linking each unit of work to the next.
- A **decentralised ownership registry** so that no single party controlled the money supply.

Bit Gold was never implemented, but it laid the conceptual foundation for Bitcoin. Many of its core ideas—computational puzzles, chained proofs, decentralised consensus—reappear in Nakamoto's design a decade later.

2.2 The Bitcoin Whitepaper

In October 2008, a person (or group) using the pseudonym Satoshi Nakamoto published a nine-page paper titled *"Bitcoin: A Peer-to-Peer Electronic Cash System"* [3]. This paper proposed the first fully functional decentralised digital currency, elegantly solving problems that had stymied researchers for decades.

2.2.1 The Problem with Traditional Systems

Nakamoto's paper begins with a simple observation: all existing electronic payment systems depend on financial institutions serving as trusted third parties. This dependence introduces several costs:

- **Transaction fees** to pay the intermediary.
- **Delays** as transactions are cleared and settled.
- **Reversibility** — transactions can be reversed by the intermediary, which increases the need for trust and introduces fraud risk.
- **Privacy concerns** — the intermediary sees all transaction details.

2.2.2 The Double-Spending Problem

The central technical challenge for any digital currency is the **double-spending problem**. Unlike a physical coin, which can only be in one place at a time, a digital token is just a string of bits that can be copied perfectly. Without a mechanism to prevent it, a dishonest user could spend the same digital coin twice—once to buy a product and again to buy something else—before anyone realises the fraud.

In centralised systems, a bank solves this trivially: it maintains a single authoritative record of everyone's balance. But in a decentralised system with no central authority, the challenge is: *how do you get everyone to agree on which transaction happened first?*

2.2.3 Bitcoin Transactions

In Bitcoin, a "coin" is not a file sitting on your computer. Instead, it is a **chain of digital signatures**. Each time a coin changes hands, the current owner digitally signs

a record that includes the hash of the previous transaction and the public key of the new owner, then appends this to the coin's history. Anyone can verify the chain of ownership by checking the signatures using the corresponding public keys.

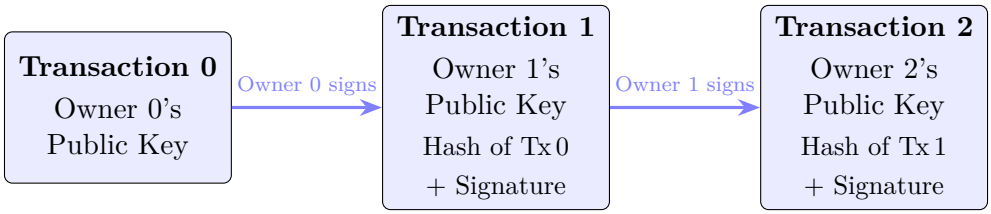

Figure 2.3: Bitcoin transaction chain. Each transaction contains the hash of the previous transaction and the new owner's public key, signed by the previous owner. This creates a verifiable chain of ownership.

Digital signatures prove ownership, but they do *not* prevent double spending on their own. A dishonest owner could sign the same coin over to two different people. To resolve this, we need a way for the entire network to agree on which transaction came first.

2.2.4 The Timestamp Server

Nakamoto's solution is to group transactions into **blocks** and pass each block through a timestamp server. The server creates a hash of the block and publishes it, linking it to the hash of the previous block. This creates a chronological chain of blocks—a *blockchain*—that establishes an indisputable order of events.

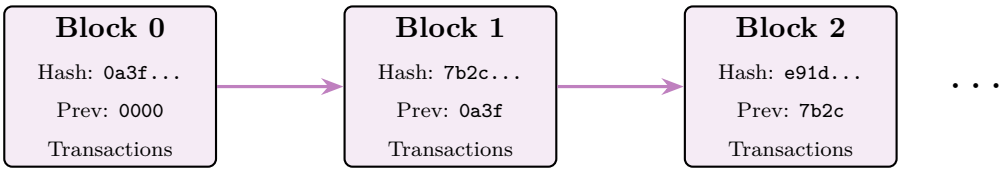

Figure 2.4: The blockchain as a timestamp server. Each block contains a batch of transactions, a hash of its own contents, and the hash of the previous block. Altering any block would change its hash and break all subsequent links.

The critical insight is this: timestamping creates an *immutable order* of transactions, which is exactly what is needed to prevent double spending. If two conflicting transactions appear, the network can now determine which one was recorded first.

2.2.5 Proof of Work

But who gets to add a new block to the chain? In a decentralised system, there is no designated authority. Nakamoto's answer is Proof of Work (PoW): a mechanism that requires participants (called *miners*) to expend significant computational effort to add a block, but makes it trivial for everyone else to verify the result.

Specifically, a miner must find a number called a **nonce** such that when the block's contents are hashed together with the nonce, the resulting hash value is below a certain target:

$$H(\text{block data} \,\|\, \text{nonce}) < \text{target} \tag{2.1}$$

Because cryptographic hash functions behave essentially as random oracles, the only way to find a valid nonce is to try values one after another—a brute-force search. This requires substantial computational work (on average), but once a valid nonce is found, anyone can verify it with a single hash computation.

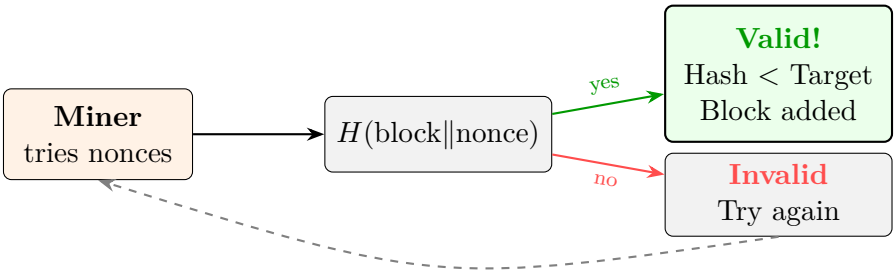

Figure 2.5: The Proof of Work mining process. A miner repeatedly tries different nonces until the hash of the block falls below the target. Finding a valid nonce is hard; verifying it is easy.

Two important rules govern how the network uses Proof of Work:

Longest Chain Rule. When multiple valid chains exist (because two miners found blocks at roughly the same time), the network accepts the chain with the most cumulative work:

$$\text{Chain}_{\text{valid}} = \arg\max_{\text{chain}} \sum_{\text{blocks}} \text{work}(\text{block}) \tag{2.2}$$

This resolves conflicts and ensures a single, globally agreed-upon transaction history.

Security Model. Let p denote the fraction of total hash power controlled by honest nodes and q the fraction controlled by an attacker. The system is secure as long as $p > q$—that is, the honest majority controls more than 50% of the network's computational power. The probability of an attacker successfully rewriting the blockchain decreases *exponentially* with the number of confirmations (subsequent blocks), making deep reorganisations practically impossible.

Chapter 3

Consensus Mechanisms and Scalability Engineering

Declaration of Contributions: This chapter was primarily authored by **Anmol Goklani**, who researched and wrote the sections on the Byzantine Generals Problem, types of consensus mechanisms (PoW, PoS, PBFT), and scalability engineering approaches (sharding, Layer 2, rollups, block size). All team members reviewed the technical content and verified the comparative tables.

The previous chapter explained *how* Bitcoin records and orders transactions. This chapter addresses two deeper questions: *how do independent nodes agree on what the truth is?* (consensus) and *how can blockchain handle more transactions?* (scalability).

3.1 The Consensus Problem

In a decentralised blockchain network, there is no central server that decides which transactions are valid and in what order they occurred. Instead, hundreds or thousands of independent nodes—each with its own copy of the ledger—must collectively agree on a single, consistent state. This is the **consensus problem**.

The difficulty is compounded by the fact that some nodes may be faulty (crashed or unresponsive) or even *malicious* (deliberately sending incorrect or contradictory information). The theoretical underpinning of this challenge is the **Byzantine Generals Problem**, formulated by Lamport, Shostak, and Pease in 1982 [8].

Definition 3.1 (Consensus Mechanism)**.** *A consensus mechanism is a protocol that enables a decentralised network of nodes to agree on a single, consistent state of the blockchain, ensuring that all honest participants validate the same transactions and maintain a common transaction history—without relying on any central authority.*

Formally, a consensus mechanism defines a state transition function:

$$C(S_t, \text{Tx}) \rightarrow S_{t+1} \tag{3.1}$$

where S_t is the current state, Tx is a set of new transactions, and S_{t+1} is the next agreed-upon state.

Any practical consensus mechanism must satisfy three key properties:

1. **Consistency** (Agreement): All honest nodes see the same state.
2. **Validity** (Integrity): Only correctly formed and properly signed transactions are accepted.
3. **Fault Tolerance** (Liveness): The system continues to operate even in the presence of faulty or malicious nodes (Byzantine resilience).

3.2 Types of Consensus Mechanisms

Different blockchain systems use different consensus mechanisms, each making different trade-offs among security, efficiency, and decentralisation. We examine the three most important ones.

3.2.1 Proof of Work (PoW)

PoW was described in detail in Section 2.2.5 as the mechanism used by Bitcoin. To summarise:

- Nodes (miners) compete to solve a cryptographic puzzle: find a nonce such that $H(\text{block}\|\text{nonce}) < \text{target}$.
- The first miner to find a valid solution broadcasts the block to the network.
- Other nodes verify the solution (a single hash computation) and accept the block if valid.
- The miner is rewarded with newly created cryptocurrency and transaction fees.

Strengths: PoW is battle-tested (Bitcoin has been running since 2009 without a successful 51% attack) and provides strong security guarantees.

Weaknesses: It is extremely *energy-intensive*. The Bitcoin network alone consumes more electricity than many countries. It also has low throughput—Bitcoin processes approximately 7 Transactions Per Second (TPS), compared to Visa's approximately 50,000 TPS.

3.2.2 Proof of Stake (PoS)

Proof of Stake (PoS) [9] replaces computational puzzles with economic incentives. Instead of miners competing with hardware, *validators* are selected to propose and validate blocks based on the amount of cryptocurrency they have "staked" (locked up as collateral).

The probability of being selected as a validator is proportional to one's stake:

$$P(\text{validator } i) \propto \text{stake}_i \tag{3.2}$$

If a validator acts dishonestly (e.g., tries to approve fraudulent transactions), their staked funds can be "slashed" (confiscated)—providing a direct financial deterrent against misbehaviour.

Strengths: PoS is vastly more energy-efficient than PoW (Ethereum reduced its energy consumption by approximately 99.95% after switching from PoW to PoS in September 2022). It also enables higher throughput.

Weaknesses: PoS can lead to *wealth centralisation*—the rich get richer, since those with more stake are more likely to be selected as validators and earn rewards.

3.2.3 Practical Byzantine Fault Tolerance (PBFT)

Practical Byzantine Fault Tolerance (PBFT) [10] is a consensus protocol designed for permissioned blockchain networks (where the set of nodes is known in advance). It works through multiple rounds of voting among nodes.

PBFT can tolerate up to f faulty (including malicious) nodes out of n total nodes, provided:

$$n \geq 3f + 1 \tag{3.3}$$

Agreement is reached when at least $2f + 1$ nodes vote for the same state.

Strengths: PBFT provides *deterministic finality*—once a block is confirmed, it is final (no possibility of reorganisation). It is also fast for small networks.

Weaknesses: PBFT does not scale well. The communication overhead grows as $O(n^2)$ with the number of nodes, making it impractical for large, open networks.

3.2.4 Comparison of Consensus Mechanisms

Table 3.1 provides a side-by-side comparison of the three consensus mechanisms.

The choice of consensus mechanism reflects a fundamental trade-off:

$$\text{Security} \longleftrightarrow \text{Scalability} \longleftrightarrow \text{Decentralisation} \tag{3.4}$$

Property	PoW	PoS	PBFT
Selection method	Computational puzzle	Stake-weighted random	Voting rounds
Energy usage	Very high	Very low	Low
Throughput	Low (∼7 TPS)	Medium–High	High (small networks)
Finality	Probabilistic	Probabilistic/Deterministic	Deterministic
Fault tolerance	> 50% honest hashpower	> 50% honest stake	$n \geq 3f + 1$
Scalability	Global, open	Global, open	Limited (permissioned)
Example	Bitcoin	Ethereum 2.0	Hyperledger Fabric

Table 3.1: Comparison of Proof of Work, Proof of Stake, and PBFT consensus mechanisms.

No existing mechanism optimises all three simultaneously. This trade-off is explored in depth in Chapter 5.

3.3 Scalability Engineering

3.3.1 The Scalability Problem

In most blockchain systems, *every node* must verify and store *every transaction*. While this redundancy is what makes blockchains secure and trustworthy, it severely limits throughput. Table 3.2 illustrates the gap.

System	Transactions Per Second (TPS)
Bitcoin	∼ 7
Ethereum (pre-merge)	∼ 15–30
Visa	∼ 50,000

Table 3.2: Throughput comparison: blockchain networks vs. traditional payment systems [1].

The goal of scalability engineering is to increase throughput without compromising security or decentralisation. Four main approaches have been proposed.

3.3.2 Sharding

Sharding divides the blockchain network into smaller groups called **shards**, each responsible for processing a subset of transactions in parallel [11].

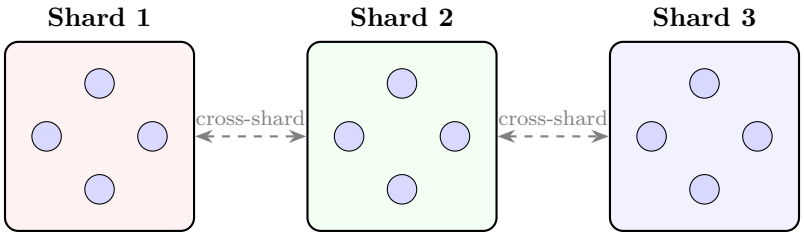

Figure 3.1: Sharding: the network is partitioned into independent shards that process transactions in parallel. Cross-shard communication enables transactions that span multiple shards.

Advantages: Throughput scales roughly linearly with the number of shards.

Challenges: Each shard has fewer nodes, which can reduce security. Cross-shard transactions require additional coordination protocols, adding complexity.

3.3.3 Layer-2 Solutions

Layer-2 solutions move transaction processing *off the main chain* (Layer 1), settling only the final result on-chain [12]. The main blockchain serves as a "court of last resort" that can resolve disputes.

Examples include:

- **Payment Channels** (e.g., Bitcoin's Lightning Network): Two parties open a channel, conduct many transactions between themselves off-chain, and only broadcast the net result to the blockchain.
- **State Channels**: A generalisation of payment channels to arbitrary state transitions.
- **Sidechains**: Independent blockchains connected to the main chain via a two-way bridge.

Advantages: Near-instant transactions with very low fees.

Challenges: Added architectural complexity; users must stay online (or use watchtowers) to prevent fraud.

3.3.4 Rollups

Rollups batch hundreds of transactions into a single transaction that is submitted to the main chain [13]. There are two varieties:

- **Optimistic Rollups:** Assume transactions are valid by default; any observer can submit a fraud proof within a challenge period if they detect an invalid transaction.

- **Zero-Knowledge (ZK) Rollups:** Each batch includes a cryptographic proof (a Zero-Knowledge Proof (ZKP)) that the batch is valid. No challenge period is needed because validity is mathematically guaranteed.

Advantages: Significant throughput improvement while inheriting the security of the main chain.

Challenges: Optimistic rollups have a delay for the challenge period; ZK rollups require complex proof generation.

3.3.5 Larger Block Size

The simplest approach to scaling is to increase the maximum size of each block, allowing more transactions per block.

Advantages: Straightforward to implement; immediate throughput improvement.

Challenges: Larger blocks take longer to propagate through the network, increasing the risk of forks. They also require more storage and bandwidth, potentially excluding nodes with limited resources and reducing decentralisation.

3.3.6 Summary of Scaling Approaches

Approach	Key Advantage	Main Challenge
Sharding	Parallel processing, linear scaling	Reduced per-shard security
Layer-2 Solutions	Off-chain speed and low fees	Architectural complexity
Rollups	Batched efficiency, inherits L1 security	Proof generation overhead
Larger Block Size	Simple implementation	Centralisation risk

Table 3.3: Summary of blockchain scalability approaches and their trade-offs.

Chapter 4

Advanced Directions and Applications of Blockchain

Declaration of Contributions: This chapter was primarily authored by **Nitin Raj**, who researched and wrote the sections on smart contract evolution, zero-knowledge proofs, cross-chain interoperability, and real-world applications across finance, supply chain, healthcare, and digital identity. All team members reviewed the content and contributed application examples.

The previous chapters established the foundations of blockchain: how blocks are linked, how transactions are validated, and how networks reach consensus. This chapter looks forward, examining the advanced technologies that are expanding blockchain's capabilities and the real-world domains where these capabilities are being applied.

4.1 Smart Contract Evolution

A **smart contract** is a self-executing program stored on a blockchain that automatically enforces the terms of an agreement when predefined conditions are met [14, 15]. Unlike traditional contracts that require lawyers and courts for enforcement, smart contracts execute automatically—the code *is* the contract.

Definition 4.1 (Smart Contract)**.** *A smart contract is a deterministic program deployed on a blockchain that takes inputs, executes predefined logic, modifies the blockchain's state, and produces outputs—all without human intervention after deployment.*

Ethereum, launched in 2015 by Vitalik Buterin [14], was the first blockchain platform to support Turing-complete smart contracts. This transformed blockchain from a simple ledger for tracking currency into a general-purpose platform for Decentralized Applications (DApps).

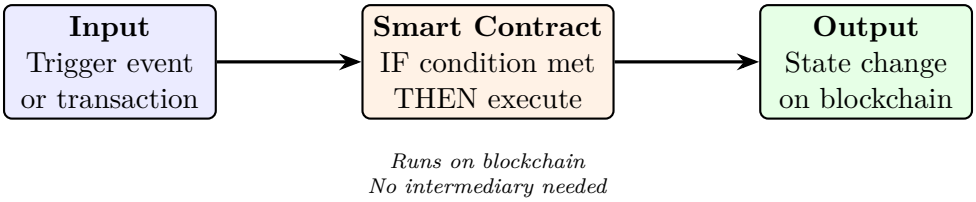

Figure 4.1: How a smart contract works. An external event triggers the contract, which evaluates its conditions and, if they are met, automatically executes the agreed-upon action and records the result on the blockchain.

4.2 Privacy Enhancements: Zero-Knowledge Proofs

One of the most exciting developments in blockchain is the application of **zero-knowledge proofs** (ZKPs) [16]. A zero-knowledge proof allows one party (the *prover*) to convince another party (the *verifier*) that a statement is true, *without revealing any information beyond the truth of the statement itself.*

Definition 4.2 (Zero-Knowledge Proof)**.** *A zero-knowledge proof for a statement x is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ satisfying:*

1. ***Completeness:*** *If x is true and $\mathcal{P}$ is honest, $\mathcal{V}$ accepts.*
2. ***Soundness:*** *If x is false, no cheating $\mathcal{P}^*$ can convince $\mathcal{V}$ (except with negligible probability).*
3. ***Zero-Knowledge:*** *$\mathcal{V}$ learns nothing beyond the validity of x.*

In blockchain, ZKPs enable **privacy-preserving transactions**: you can prove you have enough funds to make a payment without revealing your actual balance. They also power ZK-Rollups (see Section 3.3.4), enabling scalability without sacrificing verifiability.

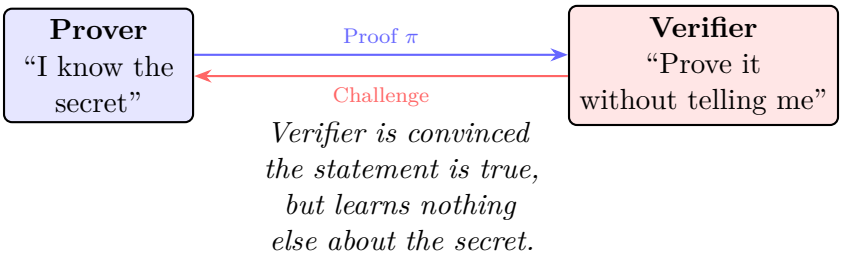

Figure 4.2: Conceptual illustration of a zero-knowledge proof. The prover demonstrates knowledge of a secret without ever revealing the secret itself.

4.3 Cross-Chain Interoperability

As the number of blockchain networks has grown—Bitcoin, Ethereum, Solana, Polkadot, and hundreds of others—a new challenge has emerged: these networks cannot natively

communicate with each other. A user holding assets on Ethereum cannot directly use them on Bitcoin's network, and vice versa.

Cross-chain interoperability refers to protocols and technologies that enable different blockchains to exchange information and value [17]. Key approaches include:

- **Atomic Swaps:** Trustless exchange of assets between two blockchains using time-locked smart contracts. Either both parties receive their assets, or the transaction is cancelled entirely.
- **Bridge Protocols:** Specialised smart contracts that lock assets on one chain and mint equivalent representations on another.
- **Relay Chains:** Dedicated blockchains (such as Polkadot's relay chain) that coordinate communication between multiple "parachains."

Cross-chain interoperability is widely regarded as essential for the long-term vision of a multi-chain ecosystem where users and applications move seamlessly between networks.

4.4 Applications of Blockchain

Blockchain's unique properties—immutability, transparency, decentralisation, and programmability (via smart contracts)—make it suitable for a wide range of applications beyond cryptocurrency.

4.4.1 Finance and Decentralised Finance (DeFi)

Blockchain's first and most mature application is in finance. Cryptocurrencies like Bitcoin provide *peer-to-peer digital cash* that can be sent anywhere in the world without banks, intermediaries, or border restrictions [3].

The rise of Decentralized Finance (DeFi) has extended this to more complex financial services: lending, borrowing, trading, and insurance—all implemented as smart contracts on platforms like Ethereum, eliminating the need for traditional financial institutions.

4.4.2 Supply Chain Management

Supply chains involve many parties (manufacturers, shippers, retailers) who often do not trust each other. Blockchain provides a shared, tamper-proof record of every step a product takes from factory to consumer [18].

Benefits include:

- **Provenance tracking:** Consumers can verify where a product came from and how it was handled.
- **Counterfeit prevention:** Each item can be assigned a unique identifier recorded on the blockchain.
- **Dispute resolution:** An immutable record reduces disagreements between parties.

4.4.3 Healthcare

In healthcare, patient data is scattered across hospitals, clinics, and insurance companies, often in incompatible formats. Blockchain can serve as a secure, interoperable layer for medical data sharing [19]:

- Patients retain *control* over who can access their records.
- Healthcare providers can access a patient's complete history (with permission) without relying on a single centralised database.
- Clinical trial data can be recorded immutably, improving transparency and trust.

4.4.4 Digital Identity

Traditional digital identity systems are controlled by governments or corporations, creating privacy risks and single points of failure. **Self-Sovereign Identity (SSI)** proposes a model where individuals own and control their own identity credentials, stored on a blockchain [20].

With SSI, you could prove your age without revealing your date of birth, or prove you have a university degree without exposing your transcript—leveraging zero-knowledge proofs (Section 4.2).

4.4.5 Summary of Applications

Domain	Blockchain Role	Key Benefit
Finance	Peer-to-peer payments, DeFi	No intermediaries, lower fees
Supply Chain	Product tracking and provenance	Transparency and anti-counterfeiting
Healthcare	Secure data sharing	Patient-controlled records
Digital Identity	Self-sovereign identity	Privacy and user autonomy

Table 4.1: Summary of blockchain applications across different domains.

Chapter 5

Challenges in Blockchain and the Blockchain Trilemma

Declaration of Contributions: This chapter was primarily authored by **Madhav Deorah**, who researched and wrote the sections on scalability issues, energy consumption, latency, transaction fees, storage bloat, and the Blockchain Trilemma. **Anmol Goklani** contributed to the explanation of the trilemma trade-off. All team members reviewed the chapter.

Blockchain technology has many useful features, but it also has some serious problems that must be solved before it can be used on a very large scale. This chapter explains these problems and then discusses the main trade-off behind many of them, known as the Blockchain Trilemma.

5.1 Scalability Issues

As discussed in Section 3.3, blockchain networks process far fewer transactions per second than traditional payment systems. Bitcoin handles about 7 TPS, Ethereum handles about 15–30 TPS, while Visa can process around 50,000 TPS [1].

This becomes a bigger problem when more people use the network. During periods of high demand, the network can become crowded, transactions take longer to confirm, and fees increase. For example, during the 2017 and 2021 cryptocurrency booms, Ethereum transaction fees sometimes went above $50–$100 per transaction.

The main reason is the basic design of many blockchains. In most systems, *every node processes every transaction.* This helps security and decentralisation, but it also limits how many transactions the network can handle.

5.2 High Energy Consumption

PoW consensus, as used by Bitcoin, requires miners to perform a huge amount of computation. The Bitcoin network's total energy use is estimated at 120–180 TWh per year, which is comparable to the annual electricity usage of countries like Argentina or Norway.

This has led to criticism from environmentalists and policymakers. Ethereum's move from PoW to PoS in September 2022, called "The Merge", showed that more energy-efficient options are possible. It reduced Ethereum's energy use by about 99.95%.

5.3 Latency and Speed

Blockchain transactions are not instant. Before a transaction is considered "confirmed", it must be:

1. Broadcast to the network.
2. Included in a block by a miner/validator.
3. Followed by several additional blocks (to reduce the risk of reorganisation).

In Bitcoin, a single block takes about 10 minutes to mine, and 6 confirmations, about 1 hour, are usually recommended for high-value transactions. Even Ethereum, with its faster 12-second block time, still needs multiple confirmations for finality.

This delay makes blockchain difficult to use for applications that need real-time payments, such as payments at a shop counter.

5.4 High Transaction Fees

Transaction fees in blockchain networks depend on demand. When many users want their transactions included in a block, they must pay higher fees to get faster confirmation. During network congestion, this creates a "fee market" where:

- Wealthy users can afford fast confirmation.
- Ordinary users may face long delays or high costs.
- Small-value transactions may no longer make sense.

This is a problem for the idea of using blockchain for *financial inclusion*, especially for providing banking services to people who do not have access to banks.

5.5 Storage Bloat and Data Availability

Another major issue in blockchain systems is **storage bloat**, which means the blockchain keeps growing over time. Since blockchains are append-only ledgers, every new block adds more data to the network. In many public blockchains, full nodes are expected to store the full transaction history. In smart-contract platforms, they may also need to store the current application state.

This design makes the blockchain easier to audit and harder to tamper with, but it also creates a storage problem. As the ledger grows, new nodes take longer to synchronise, existing nodes need more disk space, and users with weaker hardware may stop running full nodes. This can weaken decentralisation because validation may slowly become limited to organisations that can afford the storage, bandwidth, and maintenance costs [21].

This storage problem is different from normal database growth because old blockchain data cannot simply be deleted without affecting verification. A node must either keep enough data to verify the chain on its own or depend on other nodes for missing information. This creates a trade-off between:

- **Data redundancy:** Many nodes store the same data, which improves availability.
- **Storage scalability:** Nodes store less data, which reduces cost but increases dependence on other nodes.
- **Decentralisation:** Lower hardware requirements allow more people to run validating nodes.

Several methods have been suggested to reduce blockchain storage pressure. These include pruning old data, splitting data across nodes, storing large files off-chain, compression, erasure coding, and decentralised storage systems such as IPFS [22, 23]. However, each method has some drawbacks. For example, off-chain storage reduces the size of on-chain data, but the network still needs a way to make sure the external data remains available. Similarly, pruning reduces local storage needs but can make historical verification harder for new nodes.

Therefore, storage bloat is not only a hardware problem. It is also a decentralisation and data-availability problem. If storage requirements grow too quickly, the network may still work, but it may become less open to ordinary users.

5.6 The Blockchain Trilemma

The challenges discussed above are connected to each other. They show a basic trade-off first explained by Ethereum co-founder Vitalik Buterin. The **Blockchain**

Trilemma says that a blockchain system can strongly improve at most two of the following three properties at the same time [24]:

1. **Decentralisation:** The network is controlled by many independent nodes, not by one central authority.
2. **Security:** The network can resist attacks and manipulation.
3. **Scalability:** The network can process many transactions quickly and cheaply.

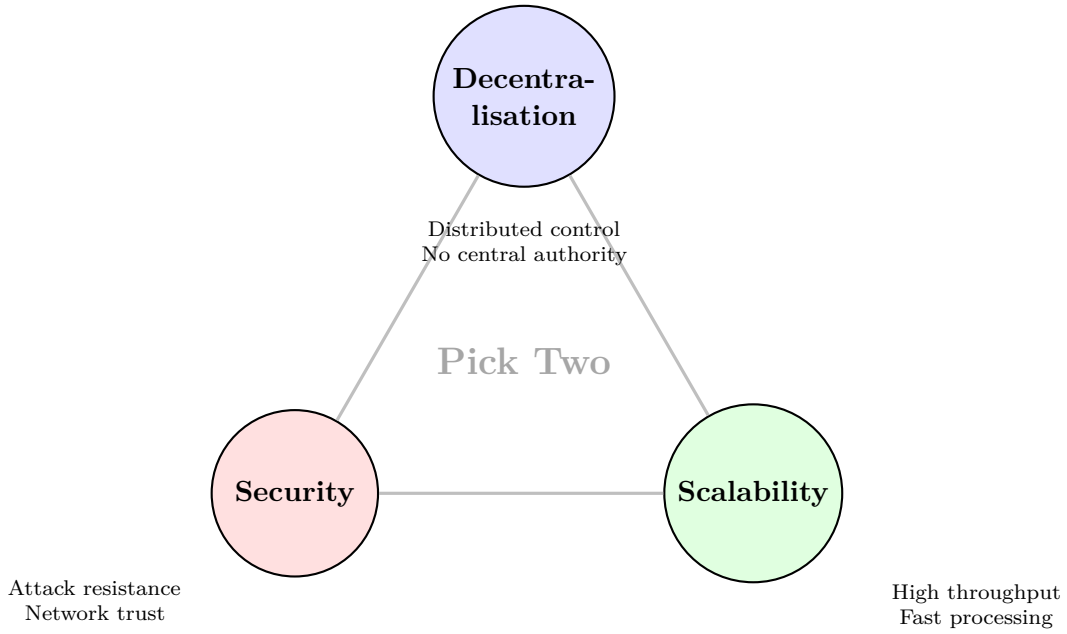

Figure 5.1: The Blockchain Trilemma. A blockchain system can usually improve only two of the three properties at the same time: decentralisation, security, and scalability. Improving one often affects another.

Table 5.1 shows how existing systems handle this trade-off.

System	Decentralised	Secure	Scalable	Trade-off
Bitcoin	✓	✓	×	Low throughput (7 TPS)
Solana	×	✓	✓	Fewer validators (centralisation)
Hyperledger	×	✓	✓	Permissioned (not open)
Ethereum 2.0	✓	✓	Improving	Still working on full scaling

Table 5.1: How different blockchain systems handle the Blockchain Trilemma.

Chapter 6

Conclusion and Future Work

Declaration of Contributions: This chapter was jointly authored by **Madhav Deorah** and **Nitin Raj**. Madhav wrote the concluding summary and Nitin wrote the future directions section. All team members contributed to the final review.

6.1 Conclusion

Blockchain technology represents a paradigm shift in how trust is established in digital systems. Instead of relying on central authorities—banks, governments, corporations—to serve as trusted intermediaries, blockchain transforms trust into a computational property: something that is *enforced by mathematics and verified by everyone.*

In this report, we have traced the intellectual journey from the earliest cryptographic time-stamping proposals of Haber and Stornetta (1991) through Nick Szabo's Bit Gold concept (1998) to Satoshi Nakamoto's landmark Bitcoin whitepaper (2008). We have examined how the core mechanisms—cryptographic hashing, Merkle trees, digital signatures, and Proof of Work—combine to create a system that is immutable, transparent, and resistant to single points of failure.

We have compared the major consensus mechanisms (PoW, PoS, PBFT) and analysed the scalability approaches (sharding, Layer-2, rollups) that aim to bring blockchain performance closer to that of traditional systems. We have surveyed the expanding landscape of blockchain applications—from finance and supply chain to healthcare and digital identity—and confronted the fundamental challenges that remain: scalability, energy consumption, latency, and the Blockchain Trilemma.

The core insight of blockchain can be summarised in a single phrase: *blockchain transforms trust from a social problem into a computational one.* Whether this transformation ultimately reshapes finance, governance, and the internet itself will depend on whether the engineering challenges we have discussed can be overcome.

6.2 Future Directions

Several promising directions are shaping the next generation of blockchain technology:

6.2.1 Scalable Blockchain Systems

The Blockchain Trilemma remains the central open problem. Research into novel sharding designs, improved rollup protocols, and hybrid consensus mechanisms continues to push the boundaries of what is achievable. The goal is a blockchain that can match the throughput of centralised systems (>10,000 TPS) while maintaining security and decentralisation.

6.2.2 Energy-Efficient Consensus

Ethereum's successful transition from PoW to PoS has demonstrated that energy-efficient consensus is viable for large-scale networks. Future work will likely explore even more efficient mechanisms, including Proof of Space-Time (used by Chia) and Proof of Authority variants for permissioned settings.

6.2.3 Cross-Chain Ecosystems

The future of blockchain is likely *multi-chain*: many specialised blockchains coexisting and interoperating seamlessly. Protocols like Polkadot, Cosmos, and cross-chain bridges are laying the groundwork for this vision, but significant challenges remain in ensuring security and usability across chain boundaries.

6.2.4 Integration with AI and IoT

Two of the most important technology trends of the coming decade—Artificial Intelligence (AI) and the Internet of Things (IoT)—are natural complements to blockchain:

- **AI + Blockchain:** AI models can be used to optimise consensus protocols, detect fraudulent transactions, and automate smart contract auditing. Conversely, blockchain can provide verifiable provenance for AI training data and model outputs.
- **IoT + Blockchain:** With billions of IoT devices generating data, blockchain can provide a decentralised, tamper-proof record of device interactions, enabling trusted machine-to-machine transactions without central servers.

6.2.5 Regulatory and Governance Frameworks

As blockchain moves from experimental technology to mainstream infrastructure, the development of clear regulatory frameworks and on-chain governance mechanisms will be essential. Decentralised Autonomous Organisations (DAOs) represent an early experiment in using blockchain for collective decision-making, but significant legal and practical questions remain.

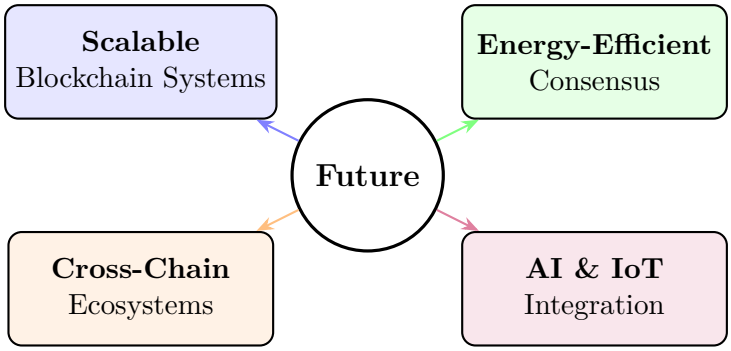

Figure 6.1: Four key future directions for blockchain technology.

Bibliography

[1] Himanshu Kohad, Santosh Kumar, and Asha Ambhaikar. Scalability issues of blockchain technology. *International Journal of Engineering and Advanced Technology (IJEAT)*, 9(3):2781–2786, 2020. doi: 10.35940/ijeat.C5305.029320.

[2] Blockchain in banking: Opportunities and challenges to overcome. `https://www.apptunix.com/blog/blockchain-in-banking-opportunities-and-challenges-to-overcome/`. Accessed: 2025-04-20.

[3] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. `https://bitcoin.org/bitcoin.pdf`, 2008. Accessed: 2025-04-20.

[4] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991. doi: 10.1007/BF00196791.

[5] Stuart Haber and W. Scott Stornetta. Improving the efficiency and reliability of digital time-stamping. pages 329–334, 1992.

[6] Nick Szabo. Bit gold. `https://nakamotoinstitute.org/library/bit-gold/`, 1998. Accessed: 2025-04-20.

[7] Ralph C. Merkle. Protocols for public key cryptosystems. pages 122–134, 1980.

[8] Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3): 382–401, 1982. doi: 10.1145/357172.357176.

[9] Sunny King and Scott Nadal. PPCoin: Peer-to-peer crypto-currency with proof-of-stake. 2012.

[10] Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. pages 173–186, 1999.

[11] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. A secure sharding protocol for open blockchains. pages 17–30, 2016. doi: 10.1145/2976749.2978389.

[12] Joseph Poon and Thaddeus Dryja. The bitcoin lightning network: Scalable off-chain instant payments. 2016.

[13] Vitalik Buterin. An incomplete guide to rollups. 2021. Accessed: 2025-04-20.

[14] Vitalik Buterin. A next-generation smart contract and decentralized application platform. 2014. Ethereum Whitepaper.

[15] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*, 2014.

[16] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. doi: 10.1137/0218012.

[17] Zibin Zheng, Shaoan Xie, Hongning Dai, Xiangping Chen, and Huaimin Wang. An overview of blockchain technology: Architecture, consensus, and future trends. *IEEE International Congress on Big Data*, pages 557–564, 2017. doi: 10.1109/BigDataCongress.2017.85.

[18] Saveen A. Abeyratne and Radmehr P. Monfared. Blockchain ready manufacturing supply chain using distributed ledger. *International Journal of Research in Engineering and Technology*, 5(9):1–10, 2016.

[19] Asaph Azaria, Ariel Ekblaw, Thiago Vieira, and Andrew Lippman. MedRec: Using blockchain for medical data access and permission management. pages 25–30, 2016. doi: 10.1109/OBD.2016.11.

[20] Christopher Allen. The path to self-sovereign identity. 2016. Accessed: 2025-04-20.

[21] Honghao Si and Baoning Niu. Research on blockchain data availability and storage scalability. *Future Internet*, 15(6):212, 2023. doi: 10.3390/fi15060212.

[22] Yehia Ibrahim Alzoubi and Alok Mishra. Techniques to alleviate blockchain bloat: Potentials, challenges, and recommendations. *Computers and Electrical Engineering*, 116:109216, 2024. doi: 10.1016/j.compeleceng.2024.109216.

[23] Yan Wang, Hao Wang, and Yanghuang Cao. Comprehensive review of storage optimization techniques in blockchain systems. *Applied Sciences*, 15(1):243, 2024. doi: 10.3390/app15010243.

[24] Pingcheng Dai et al. Understanding the blockchain scalability trilemma. *arXiv preprint arXiv:1909.00029*, 2019.

Appendix A

Supplementary Code: Simple Blockchain in Python

Declaration of Contributions: This appendix was prepared by **Nitin Raj** and **Anmol Goklani**, who jointly wrote and tested the Python implementation.

To make the concepts discussed in this report concrete, we present a minimal implementation of a blockchain in Python. This implementation demonstrates the core ideas—hashing, block linking, and proof of work—in approximately 80 lines of code.

```python
import hashlib
import json
import time

class Block:
    """Represents a single block in the blockchain."""

    def __init__(self, index, transactions, previous_hash, nonce=0):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.compute_hash()

    def compute_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
```

```python
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()


class Blockchain:
    """A simple blockchain with Proof of Work."""

    DIFFICULTY = 4  # number of leading zeros required

    def __init__(self):
        self.chain = []
        self.pending_transactions = []
        self.create_genesis_block()

    def create_genesis_block(self):
        genesis = Block(0, [], "0")
        self.chain.append(genesis)

    @property
    def last_block(self):
        return self.chain[-1]

    def proof_of_work(self, block):
        """Find a nonce such that the hash starts with
        DIFFICULTY number of leading zeros."""
        target = "0" * self.DIFFICULTY
        while not block.hash.startswith(target):
            block.nonce += 1
            block.hash = block.compute_hash()
        return block.hash

    def add_transaction(self, sender, receiver, amount):
        self.pending_transactions.append({
            "sender": sender,
            "receiver": receiver,
            "amount": amount
        })
```

```python

    def mine_block(self):
        if not self.pending_transactions:
            print("No transactions to mine.")
            return False
        new_block = Block(
            index=self.last_block.index + 1,
            transactions=self.pending_transactions,
            previous_hash=self.last_block.hash
        )
        self.proof_of_work(new_block)
        self.chain.append(new_block)
        self.pending_transactions = []
        return new_block

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]
            if current.previous_hash != previous.hash:
                return False
        return True


# --- Example Usage ---
if __name__ == "__main__":
    bc = Blockchain()
    bc.add_transaction("Alice", "Bob", 5.0)
    bc.add_transaction("Bob", "Charlie", 2.5)
    block = bc.mine_block()
    print(f"Block {block.index} mined: {block.hash}")
    print(f"Chain valid: {bc.is_chain_valid()}")
```

Listing A.1: A minimal blockchain implementation in Python.

How to run. Save the code as `blockchain.py` and execute:

```
$ python3 blockchain.py
Block 1 mined: 0000a7c3...  (hash with 4 leading zeros)
Chain valid: True
```

The `DIFFICULTY` parameter controls how many leading zeros the hash must have. Increasing it makes mining exponentially harder, illustrating the security–performance

trade-off of Proof of Work.

Appendix B

Blockchain Structure Diagram

Declaration of Contributions: This appendix was prepared by **Jai Bhadu** and **Ayushka Garg**.

Figure B.1 provides a detailed view of what a single block in a blockchain contains.

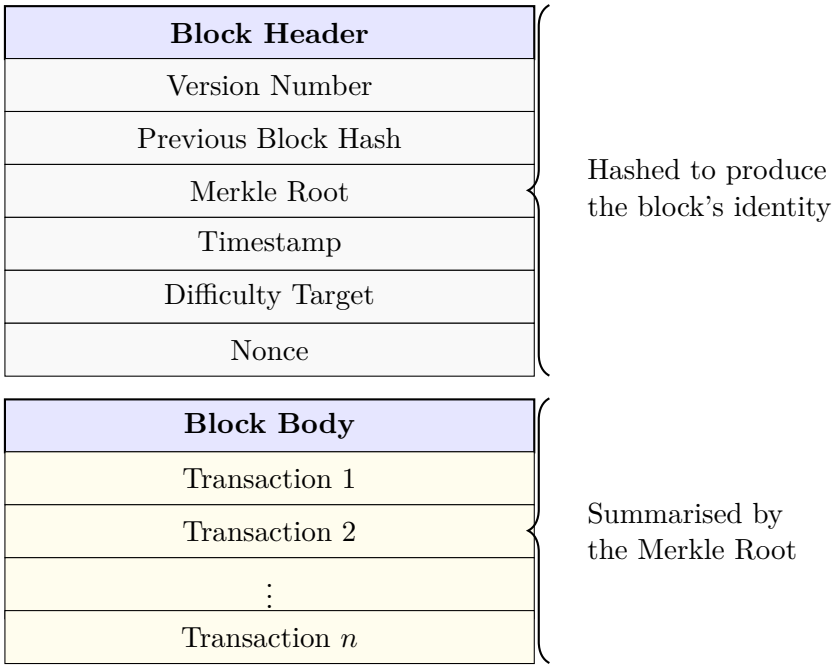

Figure B.1: Anatomy of a blockchain block. The block header contains metadata (including the hash link to the previous block and the Merkle root of all transactions). The block body contains the actual transactions.

Appendix C

Glossary of Blockchain Terms

Declaration of Contributions: This appendix was jointly prepared by all team members.

Term	Definition
Block	A data structure containing a batch of transactions, a timestamp, and a link (hash) to the previous block.
Blockchain	A chain of cryptographically linked blocks forming an immutable, distributed ledger.
Consensus	Agreement among network nodes on the valid state of the ledger.
Cryptographic Hash	A fixed-size digital fingerprint of arbitrary data; any change to the input changes the output completely.
Decentralisation	Distribution of control and data across many nodes with no single authority.
Double Spending	The fraudulent act of spending the same digital token twice.
Fork	A divergence in the blockchain when two valid blocks are produced simultaneously.
Genesis Block	The very first block in a blockchain (Block 0).
Immutability	The property that once data is recorded on the blockchain, it cannot be altered or deleted.
Merkle Tree	A binary tree of hashes used to efficiently summarise and verify large sets of data.
Mining	The process of solving a computational puzzle to add a new block (in PoW systems).
Node	A computer participating in the blockchain network.

Term	**Definition**
Nonce	A number that miners vary to find a valid block hash in Proof of Work.
Smart Contract	A self-executing program on a blockchain that enforces contractual terms automatically.
Validator	A node that validates and proposes blocks in Proof of Stake systems.
Wallet	Software that manages a user's cryptographic keys and blockchain addresses.